

SafeLine Web Application Firewall

1. Introduction

In this lab, I have built a complete cybersecurity home lab using VirtualBox, Kali Linux, Ubuntu, and SafeLine WAF. The goal here was to:

- Set up a vulnerable web application (DVWA) on Ubuntu.
- Demonstrate how to perform a basic SQL injection attack from Kali Linux.
- Show how SafeLine WAF protects against such attacks.
- Explore additional SafeLine WAF security measures (HTTP flood defense, custom deny rules, etc)

2. Basic Terms

What is a bridged network adapter?

A bridged network adapter allows a virtual machine to appear as a separate device on the same network as the host, with its own IP address and full network access.

What is DVWA?

DAM Vulnerable Web Application or DVWA is an intentionally vulnerable PHP/MySQL web application designed for practicing web security and ethical hacking in a safe, legal environment.

What is SafeLine WAF?

SafeLine WAF is a self-hosted Web Application Firewall (WAF) that acts as a reverse proxy to protect web applications from various attacks. It filters and monitors HTTP traffic to block malicious requests before they reach the backend applications creating a security barrier between applications and the Internet. It protects against common web attacks such as SQL injection, XSS, and DDoS attacks. It has semantic analysis engine that detects complex and zero-day attacks with high accuracy.

What is Docker?

Docker is an open-source containerization platform that allows you to package applications with all their dependencies into isolated units called containers. Unlike traditional virtualization, which runs a full guest OS for each application, Docker containers share the host OS kernel, making them lightweight, fast, and portable. In simple terms, think of Docker as a way to run applications inside isolated containers. Instead of installing everything directly on Ubuntu, Docker lets us run them separately. Each container has its own environment and configuration.

What is MariaDB?

MariaDB is a community-developed, commercially supported fork of the MySQL relational database management system (RDBMS) or simple terms, it is a MySQL-compatible database. It was created by the original developers of MySQL due to concerns over its acquisition by Oracle Corporation in 2009. MariaDB aims to remain free and open-source software under the GNU General Public License.

3. Prerequisites

- Host Machine with sufficient RAM (at least 8 GB recommended) and disk space (at least 50 GB free).
- Internet Connection for downloading software and updates.
- Basic Linux Command Line Knowledge (to install packages, edit configuration files, etc.).
- Virtual box (Hypervisor)
- Kali Linux (Virtual Machine 1)
- Ubuntu (Virtual Machine 2)

4. Lab Environment Setup

4.1. Download and Install VirtualBox

1. Download VirtualBox

- Go to the official VirtualBox download page:
<https://www.virtualbox.org/wiki/Downloads>
- Select the version for your host OS (Windows, macOS, Linux).

2. Install VirtualBox

- Run the downloaded installer and follow the on-screen instructions.
- Optionally, install the VirtualBox Extension Pack for additional features (USB 2.0/3.0, etc.).

4.2. Create the Kali Linux Virtual Machine

1. Download Kali Linux

- Official Kali Linux download site:
<https://www.kali.org/get-kali>
- Choose the ISO file for your architecture (64-bit is most common).

2. Create a New VM in VirtualBox

- *Name:* KaliLinux

- *Type:* Linux
- *Version:* Debian (64-bit)
- *Memory:* At least 2 GB (2048 MB)
- *Hard Disk:* Create a virtual hard disk (dynamically allocated), ~20 GB recommended.

3. Attach the Kali ISO & Install

- Go to Settings > Storage, attach the Kali ISO under the “IDE” controller.
- Start the VM, follow the Kali Linux installation steps (graphical install recommended).
- Set up a username and password you will remember (e.g., kali / kali).

4.3. Create the Ubuntu Server Virtual Machine

1. Download Ubuntu Server

- Official Ubuntu download site: <https://ubuntu.com/download/server>
- Choose the latest LTS release (e.g., 22.04 LTS).

2. Create a New VM in VirtualBox

- *Name:* UbuntuServer
- *Type:* Linux
- *Version:* Ubuntu (64-bit)
- *Memory:* At least 2 GB (2048 MB)
- *Hard Disk:* ~20 GB (dynamically allocated).

3. Attach the Ubuntu ISO & Install

- Under Settings > Storage, attach the Ubuntu ISO file.
- Start the VM, follow the Ubuntu Server installation instructions.
- Create a username/password (e.g., ubuntu / ubuntu).

4.4. Enable Bridged Networking

To have the VMs appear on the same network as your host (and each other):

1. Open VirtualBox > select your VM (e.g., UbuntuServer) > Settings > Network.
2. Adapter 1: Choose Bridged Adapter from the “Attached to” drop-down.
3. Select your host’s network interface (Ethernet or Wi-Fi).
4. Click OK.
5. Grab the IP address of the machine after you install net-tools by running the command “ifconfig” from the terminal.

6. Repeat for the Kali Linux VM as well.

5. Ubuntu Server Configuration

5.1. Initial System Updates and Installations

1. Update the Package List and Upgrade:

sudo apt-get update && sudo apt-get upgrade -y

```
vboxuser@Ubuntu240401:~$ sudo apt-get update && apt-get upgrade -y
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
E: Could not open lock file /var/lib/dpkg/lock-frontent - open (13: Permission denied)
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), are you root?
```

2. Install Net-Tools (for ifconfig and other network utilities):

sudo apt-get install -y net-tools

```
vboxuser@Ubuntu240401:~$ sudo apt-get install -y net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
net-tools is already the newest version (2.10-0.1ubuntu4.4).
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
```

3. Install OpenSSL:

sudo apt-get install -y openssl

```
vboxuser@Ubuntu240401:~$ sudo apt-get install -y openssl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssl is already the newest version (3.0.13-0ubuntu3.15).
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
```

5.2. Installing and Configuring LAMP Stack

1. Install Apache2, PHP, and MySQL:

sudo apt-get install -y apache2 php php-mysql mysql-server

```

0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
vboxuser@Ubuntu240401:~$ sudo apt-get install -y apache2 php php-mysql mysql-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mysql-server is already the newest version (8.0.46-0ubuntu0.24.04.4).
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  apache2-bin apache2-data apache2-utils libapache2-mod-php8.3 libapr1t64 libaprutil1-dbd-sqlite3
  libaprutil1-ldap libaprutil1t64 libsodium23 php-common php8.3 php8.3-cli php8.3-common php8.3-mysql
  php8.3-opcache php8.3-readline
Suggested packages:
  apache2-doc apache2-suexec-pristine | apache2-suexec-custom php-pear
The following NEW packages will be installed:
  apache2-bin apache2-data apache2-utils libapache2-mod-php8.3 libapr1t64
  libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 libsodium23 php php-common php-mysql php8.3
  php8.3-cli php8.3-common php8.3-mysql php8.3-opcache php8.3-readline
0 upgraded, 19 newly installed, 0 to remove and 22 not upgraded.
Need to get 7,121 kB of archives.
After this operation, 30.8 MB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libapr1t64 amd64 1.7.2-3.1ubuntu0.1 [
108 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libaprutil1t64 amd64 1.6.3-1.1ubuntu7
.1 [92.7 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libaprutil1-dbd-sqlite3 amd64 1.6.3-1
.1ubuntu7.1 [11.2 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 libaprutil1-ldap amd64 1.6.3-1.1ubun
t7.1 [9,120 B]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-bin amd64 2.4.58-1ubuntu8.15
[1,338 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-data all 2.4.58-1ubuntu8.15 [
163 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2-utils amd64 2.4.58-1ubuntu8.1
5 [99.7 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 apache2 amd64 2.4.58-1ubuntu8.15 [90.
2 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 php-common all 2:93ubuntu2 [13.9 kB]

```

2. Secure the MySQL Installation (optional but recommended):

`sudo mysql_secure_installation`

Follow the prompts (set a root password, remove anonymous users, disable remote root login, etc.).

```

vboxuser@Ubuntu240401:~$ sudo mysql_secure_installation

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.
The 'validate_password' component is installed on the server.
The subsequent steps will run with the existing configuration
of the component.

Skipping password set for root as authentication with auth_socket is used by default.
If you would like to use password authentication instead, this can be done with the "ALTER_USER" command
.
See https://dev.mysql.com/doc/refman/8.0/en/alter-user.html#alter-user-password-management for more info
rmation.

By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.

Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : y
Success.

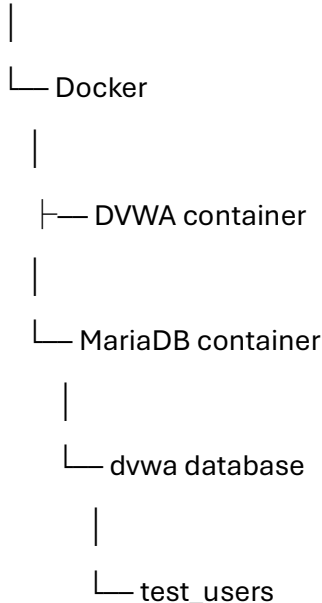
By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.

```


5.3. Installing and Configuring Damn Vulnerable Web App (DVWA) and Docker

The structure of DVWA installed in Ubuntu is as follows:

Ubuntu



1. Install Docker

sudo apt update

```
vboxuser@Ubuntu240401:~$ sudo apt update
[sudo] password for vboxuser:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,009 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,207 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,264 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,688 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [338 kB]
Get:13 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:14 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [13.1 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:16 http://in.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5,776 B]
Get:17 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [12.6 kB]
Fetched 6,608 kB in 4s (1,609 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
22 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

`sudo apt install -y ca-certificates curl`

```
vboxuser@Ubuntu240401:~$ sudo apt install -y ca-certificates curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260601~24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.13).
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
```

2. Add Docker's official GPG key:

`sudo install -m 0755 -d /etc/apt/keyrings`

```
vboxuser@Ubuntu240401:~$ sudo install -m 0755 -d /etc/apt/keyrings
vboxuser@Ubuntu240401:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
```

`ls -la /etc/apt/keyrings/`

`sudo curl -fL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc`

`ls -l /etc/apt/keyrings/docker.asc`

```
vboxuser@Ubuntu240401:~$ ls -la /etc/apt/keyrings/
total 12
drwxr-xr-x 2 root root 4096 Sep 17 10:19 .
drwxr-xr-x 9 root root 4096 Jan  2 2025 ..
-rw-r--r-- 1 root root 3817 Sep 17 10:19 docker.asc
vboxuser@Ubuntu240401:~$ sudo curl -fL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
docker.asc
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
100 3817  100 3817    0     0  54687      0 --:--:-- --:--:-- --:--:-- 55318
vboxuser@Ubuntu240401:~$ ls -l /etc/apt/keyrings/docker.asc
-rw-r--r-- 1 root root 3817 Sep 17 10:21 /etc/apt/keyrings/docker.asc
```

`sudo chmod a+r /etc/apt/keyrings/docker.asc`

```
vboxuser@Ubuntu240401:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
vboxuser@Ubuntu240401:~$ sudo tee /etc/apt/sources.list.d/docker.sources
```

3. Add the Docker repository:

`sudo tee /etc/apt/sources.list.d/docker.sources <<EOF`

`Types: deb`

`URLs: https://download.docker.com/linux/ubuntu`

`Suites: ${. /etc/os-release && echo "${UBUNTU_CODENAME:-
$VERSION_CODENAME}"}"`

`Components: stable`

Architectures: \$(dpkg --print-architecture)

Signed-By: /etc/apt/keyrings/docker.asc

EOF

```
vboxuser@Ubuntu240401:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
> Types: deb
> URIs: https://download.docker.com/linux/ubuntu
> Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
> Components: stable
> Architectures: $(dpkg --print-architecture)
> Signed-By: /etc/apt/keyrings/docker.asc
> EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
```

Then:

`sudo apt update`

```
vboxuser@Ubuntu240401:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [66.4 kB]
Hit:3 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:6 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 115 kB in 1s (93.8 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
22 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

4. Install Docker + Compose:

`sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`

These are the packages in Docker's current Ubuntu installation instructions. Ubuntu 24.04 is officially supported.


```
vboxuser@Ubuntu240401:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-ce-cli is already the newest version (5:29.8.1-1-ubuntu.24.04-noble).
containerd.io is already the newest version (2.3.5-1-ubuntu.24.04-noble).
docker-buildx-plugin is already the newest version (0.37.1-1-ubuntu.24.04-noble).
docker-compose-plugin is already the newest version (5.5.1-1-ubuntu.24.04-noble).
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  docker-ce-rootless-extras pigz
Suggested packages:
  cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  docker-ce docker-ce-rootless-extras pigz
0 upgraded, 3 newly installed, 0 to remove and 22 not upgraded.
Need to get 34.6 MB of archives.
After this operation, 141 MB of additional disk space will be used.
Get:1 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.8.1-1-ubuntu.24.04-noble [24.3 MB]
Get:2 http://in.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-extras amd64 5:29.8.1-1-ubuntu.24.04-noble [10.2 MB]
Fetched 34.6 MB in 3s (12.7 MB/s)
Selecting previously unselected package docker-ce.
(Reading database... 152368 files and directories currently installed.)
Preparing to unpack ./docker-ce_5%3a29.8.1-1-ubuntu.24.04-noble_amd64.deb...
Unpacking docker-ce (5:29.8.1-1-ubuntu.24.04-noble)...
Selecting previously unselected package pigz.
Preparing to unpack ./archives/pigz_2.8-1_amd64.deb...
Unpacking pigz (2.8-1)...
Selecting previously unselected package docker-ce-rootless-extras.
Preparing to unpack ./docker-ce-rootless-extras_5%3a29.8.1-1-ubuntu.24.04-noble_amd64.deb...
Unpacking docker-ce-rootless-extras (5:29.8.1-1-ubuntu.24.04-noble)...
Setting up docker-ce (5:29.8.1-1-ubuntu.24.04-noble)...
```

5. Verify and Test Docker

`sudo docker --version`

`sudo docker compose version`

```
vboxuser@Ubuntu240401:~$ sudo docker --version
Docker version 29.8.1, build 4a63305
vboxuser@Ubuntu240401:~$ sudo docker compose version
Docker Compose version v5.5.1
```

`sudo docker run hello-world`

```
vboxuser@Ubuntu240401:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f8b52c4d344: Pull complete
d5165e323f9c: Download complete
Digest: sha256:3b4856574ae658931675e08b04587814e4e1a127eef82f2673a91d1863a90196
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

6. Download DVWA WITHOUT Git

`cd ~`

`wget https://github.com/digininja/DVWA/archive/refs/heads/master.zip`

```
vboxuser@Ubuntu240401:~$ cd ~
vboxuser@Ubuntu240401:~$ wget https://github.com/digininja/DVWA/archive/refs/heads/master.zip
--2026-09-17 10:44:10-- https://github.com/digininja/DVWA/archive/refs/heads/master.zip
Resolving github.com (github.com)... 20.207.73.82
Connecting to github.com (github.com)|20.207.73.82|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://codeload.github.com/digininja/DVWA/zip/refs/heads/master [following]
--2026-09-17 10:44:11-- https://codeload.github.com/digininja/DVWA/zip/refs/heads/master
Resolving codeload.github.com (codeload.github.com)... 20.207.73.88
Connecting to codeload.github.com (codeload.github.com)|20.207.73.88|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1061798 (1.0M) [application/zip]
Saving to: 'master.zip.1'

master.zip.1          100%[=====>] 1.01M  5.83MB/s   in 0.2s

2026-09-17 10:44:11 (5.83 MB/s) - 'master.zip.1' saved [1061798/1061798]
```

`unzip master.zip`

```
vboxuser@Ubuntu240401:~$ unzip master.zip
Archive: master.zip
b49
  creating: DVWA-master/
  inflating: DVWA-master/.dockerignore
  creating: DVWA-master/.github/
  inflating: DVWA-master/.github/FUNDING.yml
  creating: DVWA-master/.github/ISSUE_TEMPLATE/
  inflating: DVWA-master/.github/ISSUE_TEMPLATE/bug-report---installation.md
  inflating: DVWA-master/.github/ISSUE_TEMPLATE/bug-report---vulnerability.md
  inflating: DVWA-master/.github/ISSUE_TEMPLATE/i-m-stuck.md
  creating: DVWA-master/.github/workflows/
  inflating: DVWA-master/.github/workflows/codeql-analysis.yml
  inflating: DVWA-master/.github/workflows/docker-image.yml
  inflating: DVWA-master/.github/workflows/pytest.yml
  inflating: DVWA-master/.github/workflows/shiftright-analysis.yml
  inflating: DVWA-master/.github/workflows/vulnerable.yml
  inflating: DVWA-master/CHANGELOG.md
  inflating: DVWA-master/COPYING.txt
  inflating: DVWA-master/Dockerfile
  inflating: DVWA-master/README.ar.md
  inflating: DVWA-master/README.es.md
  inflating: DVWA-master/README.fa.md
  inflating: DVWA-master/README.fr.md
  inflating: DVWA-master/README.id.md
  inflating: DVWA-master/README.it.md
  inflating: DVWA-master/README.ko.md
  inflating: DVWA-master/README.md
  inflating: DVWA-master/README.pl.md
  inflating: DVWA-master/README.pt.md
  inflating: DVWA-master/README.ru.md
  inflating: DVWA-master/README.tr.md
  inflating: DVWA-master/README.uk.md
  inflating: DVWA-master/README.vi.md
  inflating: DVWA-master/README.zh.md
  inflating: DVWA-master/SECURITY.md
  inflating: DVWA-master/about.php
  inflating: DVWA-master/compose.yml
  creating: DVWA-master/config/
  inflating: DVWA-master/config/config.inc.php.dist
```

```
mv DVWA-master DVWA
```

```
cd ~/DVWA
```

```
ls
```

```
vboxuser@Ubuntu240401:~$ mv DVWA-master DVWA
vboxuser@Ubuntu240401:~$ cd ~/DVWA
vboxuser@Ubuntu240401:~/DVWA$ ls
about.php      Dockerfile     hackable       php.ini        README.it.md   README.tr.md   security.php
CHANGELOG.md   docs           index.php      README.ar.md   README.ko.md   README.uk.md   security.txt
compose.yml    dvwa           instructions.php README.es.md    README.md       README.vi.md   setup.php
config         DVWA-master   login.php      README.fa.md   README.pl.md   README.zh.md   tests
COPYING.txt    external      logout.php     README.fr.md   README.pt.md   robots.txt    vulnerabilities
database       favicon.ico   phpinfo.php    README.id.md   README.ru.md   SECURITY.md
```

7. Start DVWA

Now simply run:

```
sudo docker compose up -d
```

This may take a few minutes the first time because Docker needs to download the DVWA and MariaDB images.

When it's finished, check:

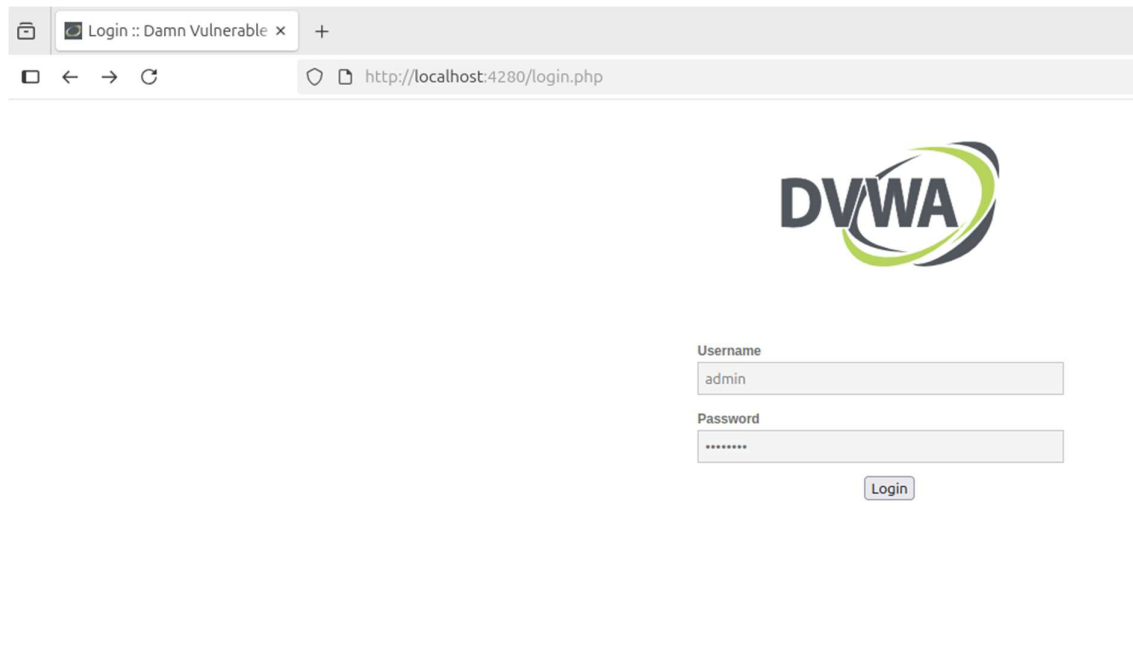
```
sudo docker compose ps
```

```
vboxuser@Ubuntu240401:~/DVWA$ sudo docker compose up -d
[+] up 37/37
✓ Image ghcr.io/digininja/dvwa:latest Pulled 68.3s
✓ Image docker.io/library/mariadb:10 Pulled 61.7s
✓ Network dvwa_dvwa Created 0.1s
✓ Volume dvwa_dvwa Created 0.0s
✓ Container dvwa-db-1 Started 4.7s
✓ Container dvwa-dvwa-1 Started 1.0s
vboxuser@Ubuntu240401:~/DVWA$ sudo docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE  CREATED          STATUS
dvwa-db-1           docker.io/library/mariadb:10        "docker-entrypoint.s..."           db        13 seconds ago  Up 8 s
econds 3306/tcp
dvwa-dvwa-1         ghcr.io/digininja/dvwa:latest      "docker-php-entrypoi..."           dvwa      9 seconds ago   Up 8 s
econds 127.0.0.1:4280->80/tcp
```

8. Open DVWA

Open Firefox/Chrome **inside your Ubuntu VM** and go to:

<http://localhost:4280>

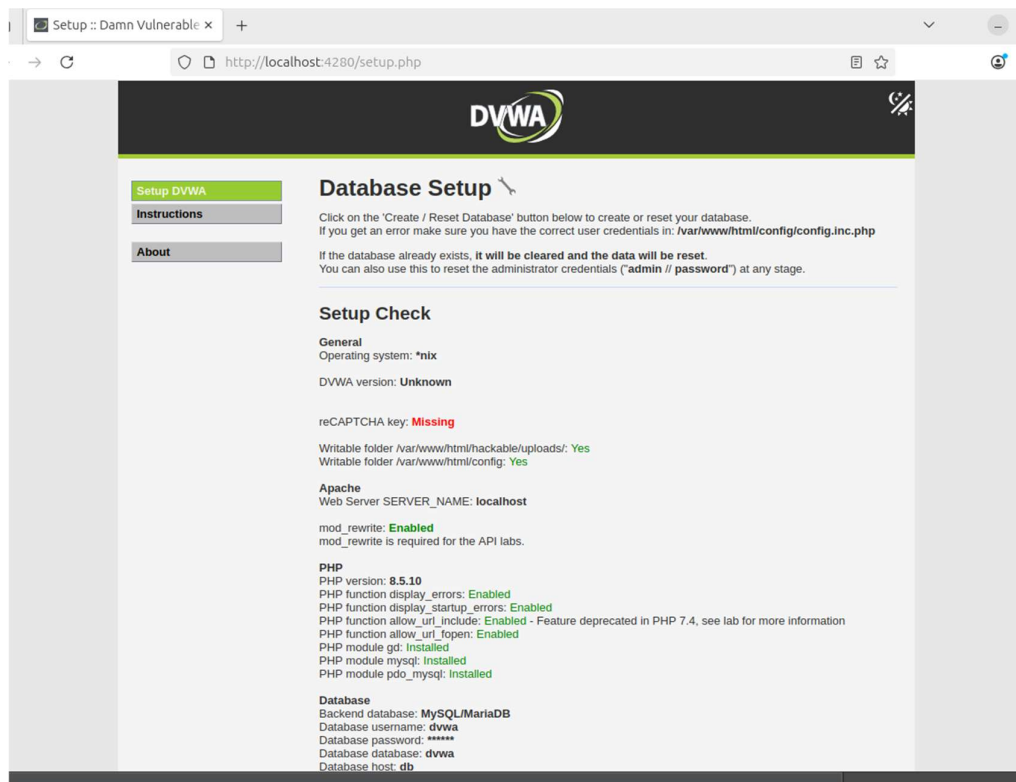


The official DVWA documentation specifies port **4280** for the Docker setup.

The default DVWA credentials are:

Username: admin

Password: password



9. Set File Permissions:

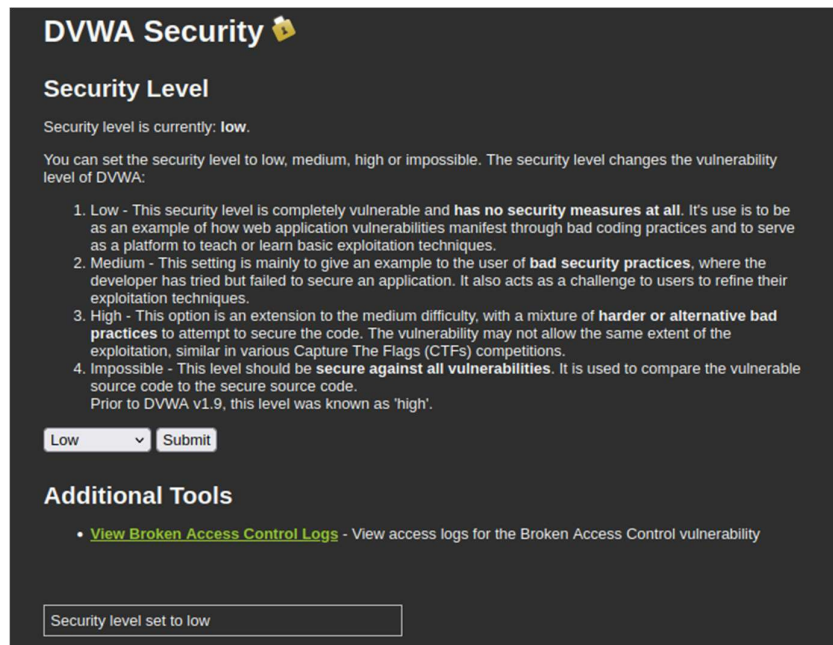
```
sudo chown -R www-data:www-data DVWA
```

```
sudo chmod -R 755 DVWA
```

```
vboxuser@Ubuntu240401:~$ sudo chown -R www-data:www-data DVWA
vboxuser@Ubuntu240401:~$ sudo chmod -R 755 DVWA
```

11. Initialize DVWA:

- Navigate to <http://<Ubuntu IP>/DVWA/setup.php> in your browser.
- Navigate to DVWA Security and set Security Level to Low.



DVWA Security 📄

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

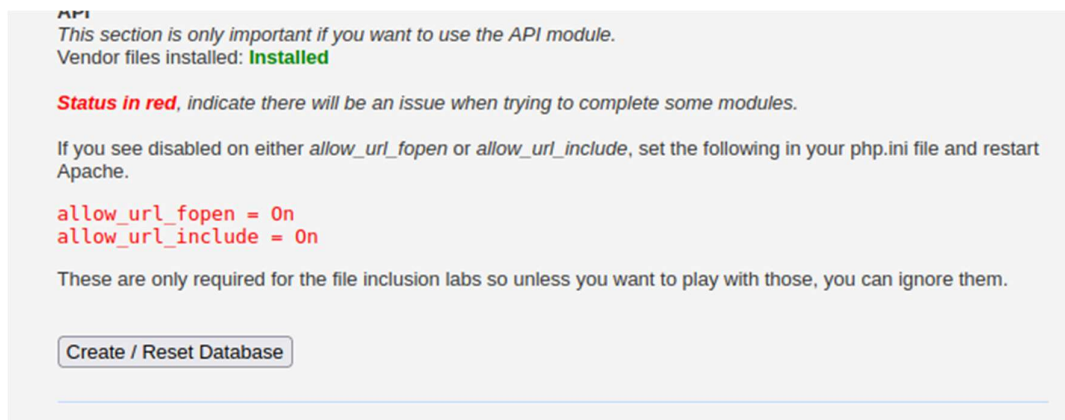
Low

Additional Tools

- [View Broken Access Control Logs](#) - View access logs for the Broken Access Control vulnerability

Security level set to low

- Navigate to Setup DVWA and click Create/Reset Database.



API

This section is only important if you want to use the API module.

Vendor files installed: **Installed**

Status in red, indicate there will be an issue when trying to complete some modules.

If you see disabled on either `allow_url_fopen` or `allow_url_include`, set the following in your `php.ini` file and restart Apache.

```
allow_url_fopen = On
allow_url_include = On
```

These are only required for the file inclusion labs so unless you want to play with those, you can ignore them.

Create / Reset Database

5.4. Adding Custom Values to the DVWA Database

You can add custom data to the DVWA database to demonstrate SQL injection:

1. Go to your DVWA directory

```
cd ~/DVWA
```

2. Check your Docker containers

```
sudo docker compose ps
```

```
sudo docker compose exec db mariadb -u dvwa -pp@ssw0rd dvwa
```

```
vboxuser@Ubuntu240401:~$ cd ~/DVWA
vboxuser@Ubuntu240401:~/DVWA$ sudo docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
dvwa-db-1	docker.io/library/mariadb:10	"docker-entrypoint.s..."	db	2 hours ago	Up 2 hour
s 3306/tcp					
dvwa-dvwa-1	ghcr.io/digininja/dvwa:latest	"docker-php-entrypoi..."	dvwa	2 hours ago	Up 2 hour
s 127.0.0.1:4280->80/tcp					

```
vboxuser@Ubuntu240401:~/DVWA$ sudo docker compose exec db mariadb -u dvwa -pp@ssw0rd dvwa
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 19
Server version: 10.11.19-MariaDB-ubu2204 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [dvwa]> create table test_users(
```

3. Add a sample table/entries:

```
CREATE TABLE test_users (
    id INT NOT NULL AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    password VARCHAR(50) NOT NULL,
    PRIMARY KEY (id)
);
```

```
MariaDB [dvwa]> create table test_users(
-> id int not null AUTO_INCREMENT,
-> username VARCHAR(50) not null,
-> password VARCHAR(50) not null,
-> primary key(id)
-> );
Query OK, 0 rows affected (0.034 sec)
```

```
INSERT INTO test_users (username, password) VALUES
```

```
('alice', 'alice123'),
```

```
('bob', 'bob123'),  
('admin', 'admin123');  
  
exit;
```

```
MariaDB [dvwa]> insert into test_users(username, password) values ('alice','alice123'), ('bob','bob123')  
, ('admin','admin123');  
Query OK, 3 rows affected (0.010 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
MariaDB [dvwa]> exit;  
Bye
```

6. Installing and Configuring SafeLine WAF on Ubuntu

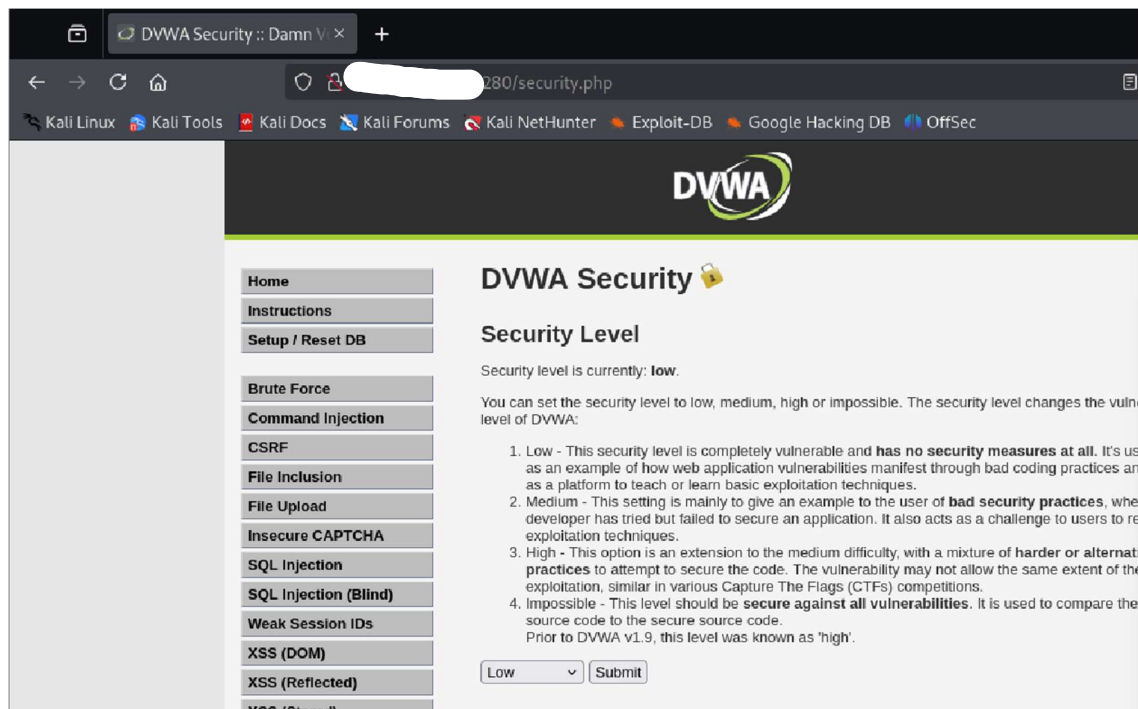
Head to the following website <https://cyberserval.tech/landing/safeline> and select “Install”. SafeLine WAF typically provides an automatic install script or a manual install method, follow the steps below for the automatic install:

- Follow the below command on your Ubuntu CLI –
`cd ~`
`curl -fLk https://waf.chaitin.com/release/latest/setup.sh -o safeline-setup.sh`
`sudo bash safeline-setup.sh`
 - Follow On-Screen Prompts to complete installation.
 - You will be provided with an admin password and username as well as the URL to access Safeline, typically on port 9443
-
- Open the SafeLine dashboard on Ubuntu: <https://<Ubuntu ip>:9443>

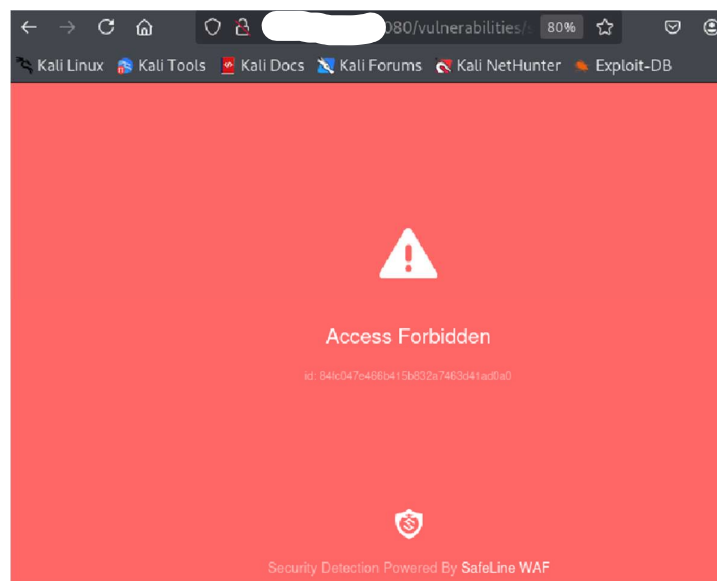
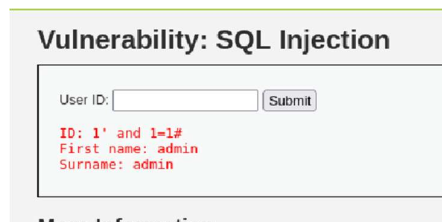
7. Demonstrating SQL Injection from Kali Linux

7.1. Launching the Attack

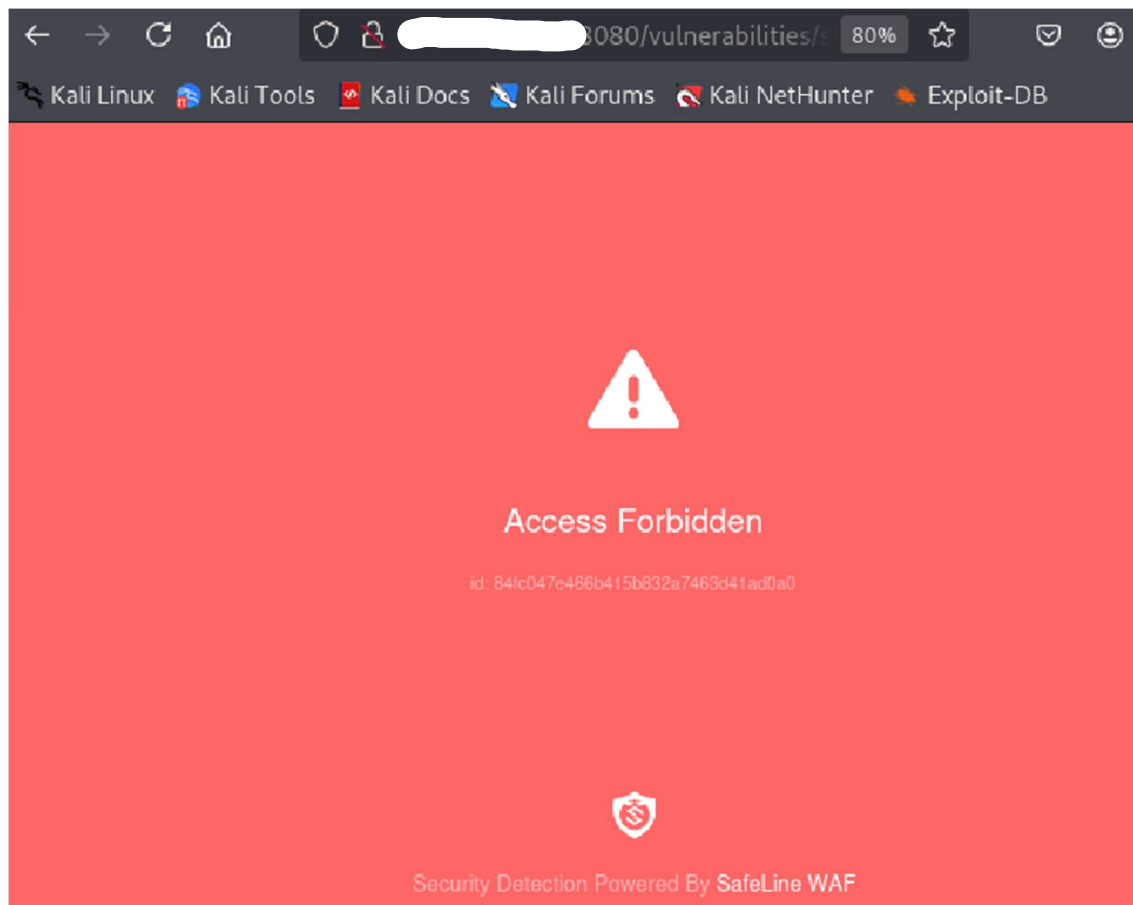
- Open Kali Linux and browse to the DVWA site (This will now be redirected to https): <http://<UbuntuIP>:8080/login.php>
- Log In to DVWA (default credentials in DVWA are admin / password unless changed).
- Set DVWA Security Level to low (in the DVWA Security tab).



- Go to SQL Injection Section and try typical SQL injection strings (e.g., admin' OR '1'='1).
- The moment we enter the sql injection (1' and 1=1#), SafeLine WAF blocks it



- We can try using a different injection like 1' OR '1'='1'#, we get the same result



7.2. Observing SafeLine WAF Protection

- Navigate to 'Applications' in SafeLine WAF and click on 'Add Application'
- In our Lab, port 8080 is SafeLine's listening port and port 4280 is DVWA's actual port
- Put the upstream as *http://<Ubuntu ip>:4280*

The screenshot shows the 'Add Application' configuration page in the SafeLine WAF interface. The 'Domain' field contains 'www.example.com, support *'. The 'Port' field is set to '8080'. The 'Upstream' field is set to 'http://[redacted]:4280'. The 'Reverse Proxy' option is selected. There are buttons for 'ADD LISTENING PORT', 'ADD UPSTREAM', 'CANCEL', and 'SUBMIT'.

- The structure looks somewhat like this



SafeLine WAF should detect and block malicious injection attempts. Look for WAF logs to see blocked SQL injection attempts. You can also see error or block pages if the WAF is in blocking mode.

SafeLine WAF > Attacks

Semantic Analysis

Semantic analysis identifies risky requests such as SQL injection, XSS, and command injection, helping uncover attacks that bypass traditional rules.

- Observe: log risky requests only, without blocking.
- Balanced protection: block obvious attacks while minimizing false positives.
- High-intensity protection: block more risky requests, with a relatively higher false-positive rate.

Policy changes apply only to enabled detection modules in the current global semantic analysis configuration.

[GO TO CONFIGURATION](#) [DONT SHOW THIS AGAIN](#)

Events LOGS

IP Addr Domain Port Start At End At Auto refresh Off REFRESH

IP Addr	Applications	Attack Count	Duration	Start At
LAN IP	http/	80/	3	4 minutes
				2026-09-20 15:40:54

20 per page, total 1

SafeLine WAF > Applications

1 Applications Application

DVWA [DETAIL](#)

DEFENSE

Domain: * Port: 8080

ROS TD 18 BLK TD 3

HTTP F... BOT PR... AUTH [ATTACKS](#)

SafeLine WAF > Statistics

143/statistics

Statistics

Requests 21 Views (PV) 10 Visitors(UV) 1 Unique IP 3 Blocked 3 IP Addr 1

4xx Errors 1 Error Rate 4.76% 4xx Blocked 3 Blocked Rate 14.29% 5xx Errors 0 Error Rate 0.00%

Geo Location

Requests Status Max 21

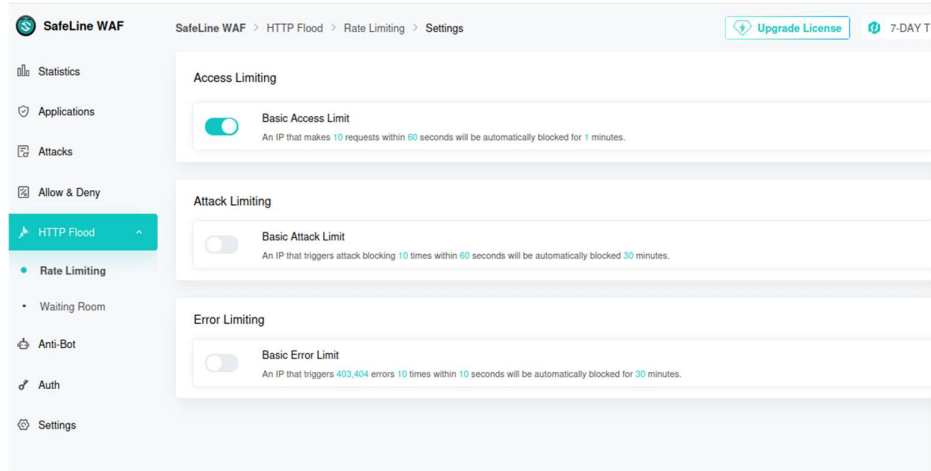
Blocking Status Max 3

Version 9.4.1

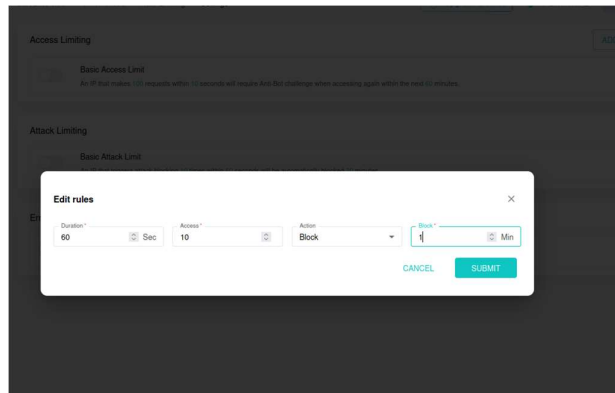
8. SafeLine WAF Advanced Configurations

8.1. HTTP Flood Defense

- Enable HTTP Flood/DoS settings in SafeLine WAF.



- Configure Thresholds (requests per second) and Penalty or Ban duration.



- Test by sending multiple requests from Kali (e.g., using ab, siege, or other load tools): `ab -n 100 -c 10 http://<ubuntu ip>:8080/login.php`

```
kali-linux-2024.4-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

File Actions Edit View Help

$ ab -n 100 -c 10 http://192.168.29.127:8080/login.php
This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 192.168.29.127 (be patient).....done

Server Software:
Server Hostname:
Server Port:

Document Path: /login.php
Document Length: 1342 bytes

Concurrency Level: 10
Time taken for tests: 3.732 seconds
Complete requests: 100
Failed requests: 0
Total transferred: 207000 bytes
HTML transferred: 134200 bytes
Requests per second: 26.80 [#/sec] (mean)
Time per request: 373.199 [ms] (mean)
Time per request: 37.320 [ms] (mean, across all concurrent requests)
Transfer rate: 54.17 [Kbytes/sec] received

Connection Times (ms)
           min      mean[+/-sd] median   max
Connect:    0        1  1.0+0.4        1    4
Processing:  5      226 709.6+177    17    3384
Waiting:     5      226 709.6+177    16    3384
Total:       6      228 709.6+177    18    3385

Percentage of the requests served within a certain time (ms)
 50%    18
 66%    26
 75%    33
 80%    36
 90%    65
 95%   2538
 98%   3370
 99%   3385
100%   3385 (longest request)
```

- Check SafeLine if it has blocked the requests or not

SafeLine WAF > HTTP Flood > Rate Limiting

By limiting the rate of requests from a single source, it effectively defends against DDoS attacks, malicious crawlers and bots.

Rate Limiting

Illustration: "No rate limit" vs "With rate limit" showing traffic being blocked.

IP Addr	Application	Start At	End At	UNBLOCK ALL	SETTINGS
LAN IP	DVWA	Reason: 10 Reqs within 60 seconds, Basic Access Limit was triggered Action: Block 1 minutes	Blocked	2026-09-20 18:15:00	Unblock
LAN IP	DVWA	Reason: 10 Reqs within 60 seconds, Basic Access Limit was triggered Action: Block 1 minutes	Blocked	2026-09-20 18:13:50	Unblocked

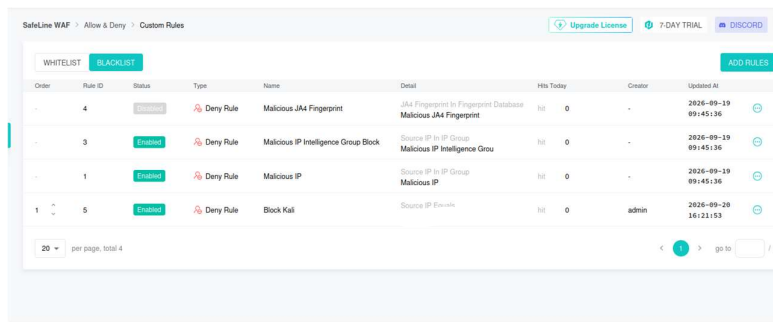
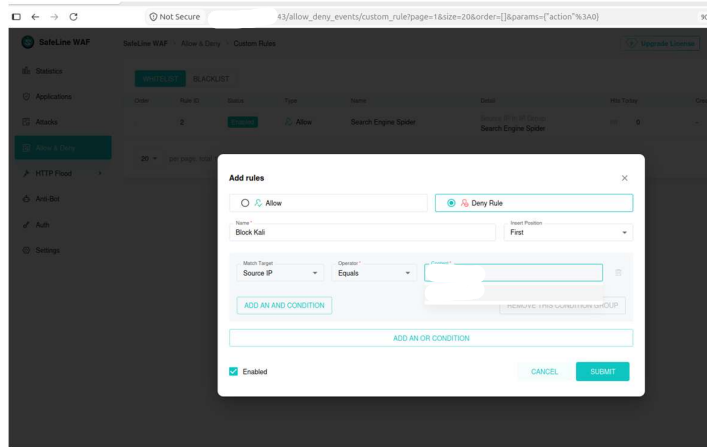
20 per page, total 2

Version 9.4.1

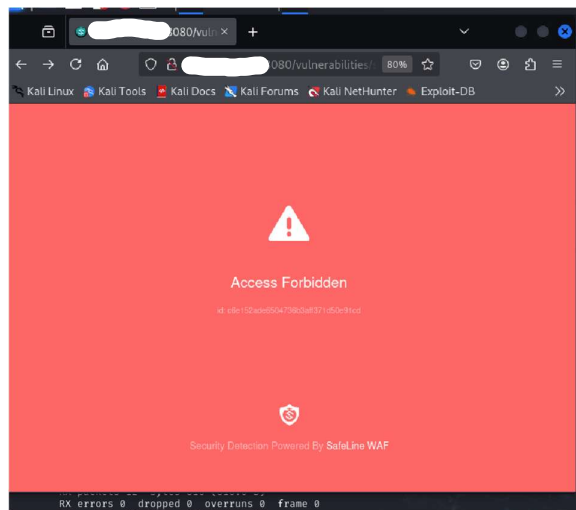
8.3. Custom Deny Rules (Blocking Kali IP)

- Identify the IP of your Kali VM (e.g., 192.168.1.50).
- Add a Deny Rule in SafeLine WAF:

- **Match Source IP:** 192.168.1.50
- **Action:** Block or Deny.



- **Test from Kali:** You should receive a blocked response once you refresh DVWA in Kali Linux.



9. Conclusion

This project provided hands-on experience in deploying and configuring SafeLine WAF to protect a web application against common web-based attacks. The lab involved

configuring WAF security rules, monitoring HTTP traffic, detecting malicious requests such as SQL injection and implementing HTTP flood protection through rate limiting and IP blocking. Controlled testing from Kali Linux demonstrated how SafeLine can identify and mitigate suspicious and excessive HTTP requests before they reach the backend application. Overall, the project strengthened practical skills in web application security, traffic analysis, attack detection, WAF configuration, and security event monitoring.